

Sakthy re-learns DB stuff properly

December 4, 2025

Contents

1 Motivation

I know SQL but do I really **know** SQL? I have been able to get by with basic queries and I even know how advanced concepts like indexing, normalization, and transactions. However, I feel like my understanding is superficial. I want to understand how it works under the hood and when to use what. I have debugged numerous issues in the past but when I think about them now, I can't seem to recall the exact details. So this document is my personal journey to re-learn and deeply understand database concepts.

2 Understanding SQL query cost

Why do we need optimized SQL queries? Why can't we just write any query and let the database handle it? So it seems that there seems to be a cost associated with executing SQL queries. This cost can be measured in terms of time taken to execute the query, resources consumed (like CPU and memory), and the impact on other operations happening in the database.

From what I have experienced so far, the cost of SQL queries is made up of (within things that you can control):

- **Data Volume:** Duh. The more data you have to process, the longer it takes.
- **Query Complexity:** If an SQL query is doing way too much work in a single statement, it can be slow.
- **Index Usage:** Lack of indexes they say but it's not that simple. Double edged sword. Indexes speed up read operations but can slow down write operations.
- **Locking and Concurrency:** When we are running multiple queries at the same time, they can interfere with each other. Locks and delays can add to the cost.
- **Network Latency:** If the database is on a different server, the time taken to send and receive data over the network can add to the overall cost. Usually bottleneck when the connection pool is exhausted sometimes.

2.1 Case 1: EXPLAIN ANALYZE statement

I have seen the **EXPLAIN ANALYZE** statement in SQL but do I understand it properly? *Heck no.* So I'm stuck in a solution where I have to optimize queries but I don't know how to do it properly. Well I have written a monstrosity of a query to export data from MySQL to CSV files. It works and accomplishes the task.

```
1 WITH RECURSIVE city_items AS (  
2     SELECT  
3         lr.id AS location_restriction_id,  
4         lr.description,  
5         'origin_multiple_cities' AS field_name,  
6         lr.origin_multiple_cities AS original_cities_csv,  
7         TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(lr.origin_multiple_cities, ',',  
8             numbers.n), ',', -1)) AS item_value,  
9         numbers.n  
10    FROM  
11    rate_card_locationrestriction lr  
12    CROSS JOIN (  
13        SELECT  
14            numbers.n  
15        FROM  
16            numbers
```

```

12      SELECT 1 AS n UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL
13      SELECT 4 UNION ALL SELECT 5
14      UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL
15      SELECT 9 UNION ALL SELECT 10
16      UNION ALL SELECT 11 UNION ALL SELECT 12 UNION ALL SELECT 13 UNION
17      ALL SELECT 14 UNION ALL SELECT 15
18      UNION ALL SELECT 16 UNION ALL SELECT 17 UNION ALL SELECT 18 UNION
19      ALL SELECT 19 UNION ALL SELECT 20
20  ) numbers
21  WHERE
22      lr.origin_multiple_cities IS NOT NULL
23      AND lr.origin_multiple_cities != ''
24      AND numbers.n <= 1 + (LENGTH(lr.origin_multiple_cities) - LENGTH(
25          REPLACE(lr.origin_multiple_cities, ',', '')))
26  )
27  SELECT
28      si.location_restriction_id,
29      si.description,
30      c.id AS company_id,
31      c.company_name,
32      rc.id AS rate_card_id,
33      rc.name AS rate_card_name,
34      si.field_name,
35      si.item_value,
36      si.original_cities_csv,
37      CASE
38          WHEN COUNT(DISTINCT s1.id) > 0 OR COUNT(DISTINCT s2.id) > 0 THEN '
39          Yes'
40          ELSE 'No'
41      END AS found_in_lookup,
42      COALESCE(GROUP_CONCAT(DISTINCT s1.id ORDER BY s1.id SEPARATOR ','), '')
43      ) AS matched_by_name_ids,
44      COALESCE(GROUP_CONCAT(DISTINCT s2.id ORDER BY s2.id SEPARATOR ','), '')
45      ) AS matched_by_code_ids
46  FROM
47      city_items si
48  LEFT JOIN
49      rate_card_ratecard_location_restrictions rclr ON rclr.
50      locationrestriction_id = si.location_restriction_id
51  LEFT JOIN
52      rate_card_ratecard rc ON rc.id = rclr.ratecard_id
53  LEFT JOIN
54      company_company c ON c.id = rc.shipper_company_id
55  LEFT JOIN
56      common_cityv2 s1 ON LOWER(s1.name) = LOWER(si.item_value)
57  LEFT JOIN
58      common_cityv2 s2 ON LOWER(s2.name_wo_diacritics) = LOWER(si.item_value
59      )
60  WHERE
61      TRIM(si.item_value) != '' and rc.shipper_company_id = '572'
62  GROUP BY
63      si.location_restriction_id,
64      si.description,
65      c.id,

```

```

56     c.company_name ,
57     rc.id ,
58     rc.name ,
59     si.field_name ,
60     si.item_value
61 ORDER BY
62     c.company_name ,
63     rc.name ,
64     si.location_restriction_id;

```

Sorry for abusing your eyes with this monstrous query. What this query does is that it extracts individual city names from a comma-separated list of cities stored in the `origin_multiple_cities` field of the `rate_card_locationrestriction` table. It then checks if these city names exist in the `common_cityv2` table either by name or by name without diacritics. Finally, it aggregates the results to show which cities were found and their corresponding IDs. The query seems bad but how bad it is?

2.1.1 Analyzing the EXPLAIN output

So I know what it does, but what's the damage? So hence, let's run `EXPLAIN ANALYZE` on this query. The output is a below:

```

-> Sort: c.company_name, rc.'name', si.location_restriction_id
    (actual time=1593..1593 rows=4 loops=1)
    -> Stream results (actual time=1593..1593 rows=4 loops=1)
        -> Group aggregate: group_concat(distinct common_cityv2.id ...)
            (actual time=1593..1593 rows=4 loops=1)
            -> Sort: si.location_restriction_id, si.'description', ...
                (actual time=1593..1593 rows=43 loops=1)
                -> Stream results (cost=62.3e+12 rows=623e+12)
                    (actual time=1406..1593 rows=43 loops=1)
                    -> Left hash join (lower(s2.name_wo_diacritics)=...)
                        (cost=62.3e+12 rows=623e+12)
                        (actual time=1406..1593 rows=43 loops=1)
                        -> Left hash join (lower(s1.'name')=...)
                            (cost=70.5e+6 rows=705e+6)
                            (actual time=616..813 rows=13 loops=1)
                            -> Filter: (trim(...) <> '' and ...)
                                (cost=577 rows=797)
                                (actual time=1.59..1.66 rows=4 loops=1)
                                -> Inner hash join (no condition)
                                    (cost=577 rows=797)
                                    (actual time=1.58..1.6 rows=80 loops=1)
                                    -> Table scan on numbers
                                        (actual time=0.0314..0.0339 rows=20)
                                    -> Hash
                                        -> Nested loop inner join
                                            (actual time=0.185..1.51 rows=4)
                                            -> Filter: origin_multiple_cities

```

```

                                IS NOT NULL
                                (actual time=0.00221..0.00221
                                rows=0.0132 loops=302)
    -> Hash
        -> Table scan on s1
            (cost=122 rows=884073)
            (actual time=0.118..270 rows=890564)
    -> Hash
        -> Table scan on s2
            (cost=10.9 rows=884073)
            (actual time=0.0835..275 rows=890564)

```

So the output is tree-structure like and needs to be read from innermost to outermost. Some key metrics to remember are:

- **actual time:** Time in milliseconds
 - Format: `actual time=start..end`
- **rows:** How many rows were processed
- **cost:** Estimated abstract unit for estimating query expensive
- **loops:** How many times this operation ran

Understanding "cost" Cost is a weighted sum of various operations. For example, MySQL calculates it like this.

$$\begin{aligned}
 \text{cost} = & (\text{rows_examined} \times \text{row_eval_cost}) \\
 & + (\text{pages_read} \times \text{io_block_read_cost}) \\
 & + (\text{index_lookups} \times \text{key_lookup_cost}) + \dots
 \end{aligned}$$

where each cost as a weight associated to it. So the "cost" is relative. It has no real value. But it abstractly show you how expensive an operation is.

So what does this mean for my query? The flow seems to be like this:

1. Scans 890K cities twice (540ms)
2. Builds hash tables for lookups (616-1406ms)
3. Processes only 4 location restrictions that match the filter
4. Returns 43 intermediate rows, grouped down to 4 final rows
5. Total execution time: 1.6 seconds

So these kinda reveal some serious issues:

- No index usage because of `LOWER()` functions on both sides of joins

- Full table scans on a 890K row table (twice!)
- Plan shows 62.3e+12 cost (12 trillion!)
- The actual data needed is tiny (4 restrictions, 4 cities) but MySQL doesn't know this upfront and must scan everything

So **EXPLAIN ANALYZE** statement can kinda give you understanding how good or bad query is (*although I do not yet understand it fully, just minimal understanding alone can give you so much context*). For my use-case, this is an ad-hoc script that I'm gonna run once to generate a CSV report. So I don't need to fix this. Yay! *PS: Please do fix this if its a query that you will run more than once.*